Unidad académica multidisciplinaria
Mante -centro

Mecánicas , ondas mecánicas y sonido
práctica arduino_base

Equipos

Jimena Jiménez Guererro
Allison Evelyn Mata Rivera
Osmar Alvarado Ávalos
Juan Pablo Suárez González
Paulina ambriz reynaga
Adriana Guadalupe Cruz Castillo
Yudith Medina Rodríguez

MATERIA: MECANICA ONDAS MECANICAS Y SONIDO

DOCENTE: DANIEL LOPES PIÑA

GRUPO Y GRANDO: 1 E

28/sep/2026

• Intermitencia

Explicación

Ese es tu práctica de *Arduino UNO - LED parpadeando*, es la práctica 1 de todos los de Sistemas.

Te explico lo que se ve en tu video:

1. Lo físico (Hardware)

- *Placa azul:* Es un Arduino UNO, es como el cerebro. Le manda energía y órdenes al LED.

- *Cable azul transparente:* Es el cable USB, por ahí le pasas el programa desde tu laptop Dell y también le da corriente.

- *LED blanco/azul prendido:* Está conectado con 2 cables (azul y negro) al pin 13 y a GND (tierra) del Arduino. Ese LED es el que estás controlando.

2. Lo que hace tu código (lo que se ve en tu laptop)

Es el código clásico `Blink`:

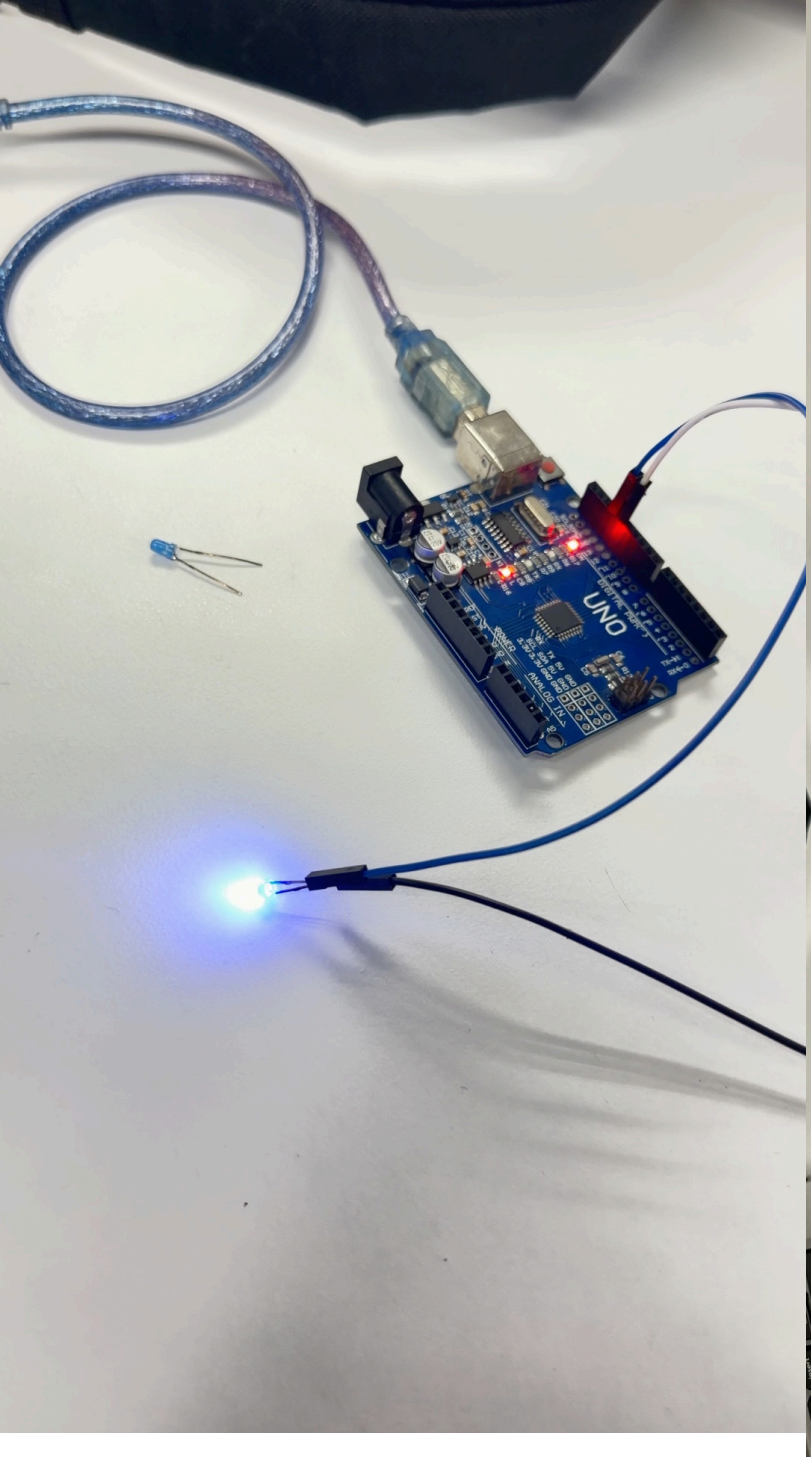
```
void setup() {
pinMode(LED_BUILTIN, OUTPUT); // Le dices al pin 13 que va a ser salida de luz
}

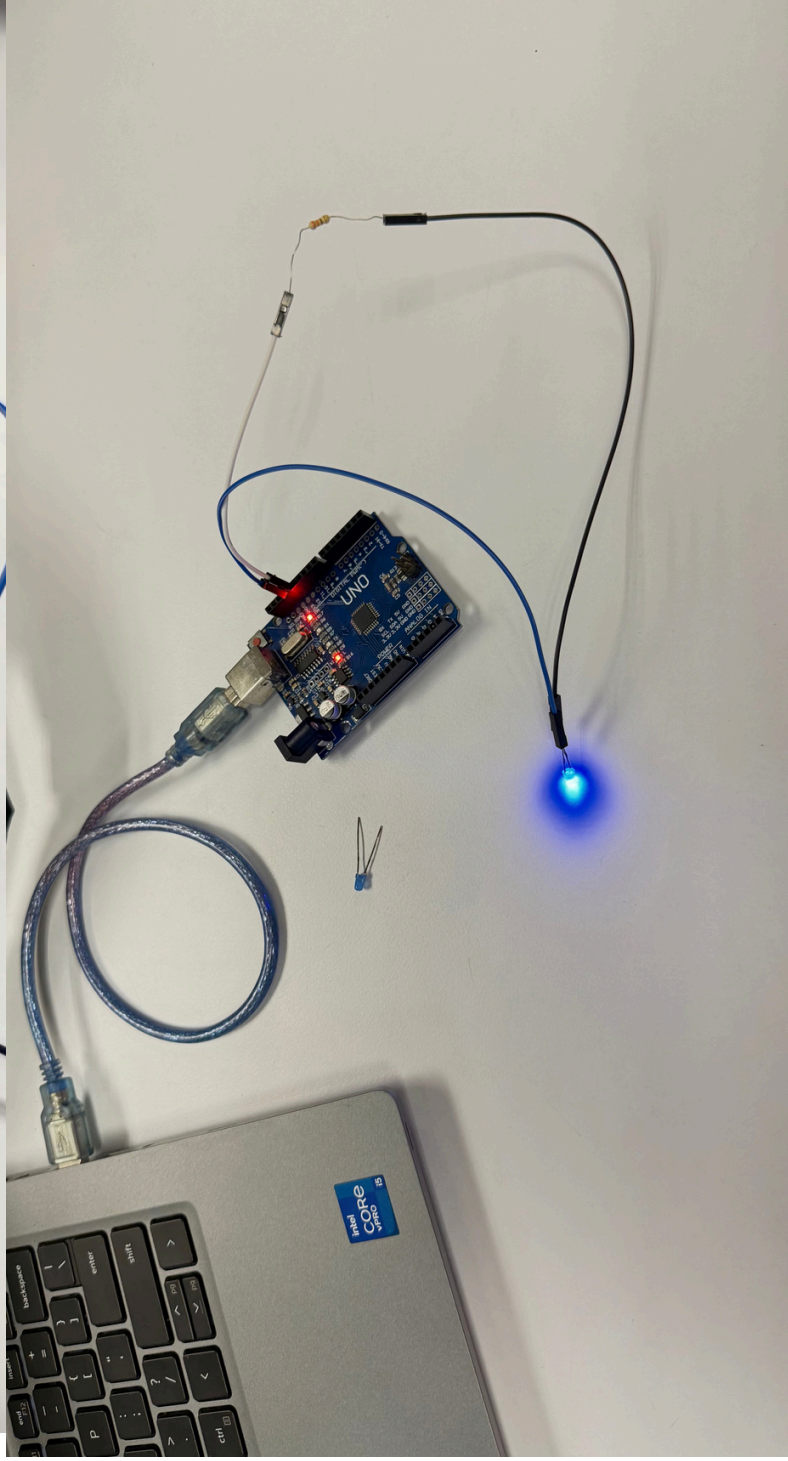
void loop() {
digitalWrite(LED_BUILTIN, HIGH); // Prende el LED (le manda voltaje)
delay(1000); // Espera 1 segundo (1000 milisegundos)
digitalWrite(LED_BUILTIN, LOW); // Apaga el LED (le quita el voltaje)
delay(1000); // Espera 1 segundo
}
```

Y eso se repite para siempre en `loop()` -> prende 1 seg, apaga 1 seg. Por eso en tu video el LED parpadea.

Para tu reporte escribe esto:

> "En esta práctica se conectó un LED externo al pin 13 del Arduino UNO. Se cargó el programa Blink desde el IDE de Arduino, el cual configura el pin como salida y alterna entre estado alto (HIGH) y bajo (LOW) con un retardo de 1000 ms, logrando el parpadeo del LED. Esto demuestra el principio básico de control de salidas digitales en sistemas embebidos."


UNO


UNO
intel
CORE

• Atenuación

Explica

Este ya es el nivel 2 del anterior. Ya no es parpadeo fijo, ahora el LED parpadea *según lo que tú le digas con la perilla.*

Lo que se ve en tu video e imagen:

1. El circuito
- *El Arduino de abajo* es el que hace todo. Está conectado a tu laptop.
- *La perillita con 3 cables (amarillo, blanco, negro):* Es un *potenciómetro*. Es como un control de volumen. Cuando lo giras, cambia la resistencia y manda diferente voltaje.
- *El cable azul y negro con el LED:* Es la salida. Ese LED es el que va a parpadear.

Conexión: El potenciómetro va a `5V - GND - A0`. El LED va al pin 13.

2. El código que se ve en tu captura

Esto es lo que hace:

sensorValue = analogRead(A0); // 1. Lee el valor de la perilla (0 a 1023)

digitalWrite(LED_BUILTIN, HIGH); // 2. Prende el LED
delay(sensorValue); // 3. Se queda prendido el tiempo que diga la perilla

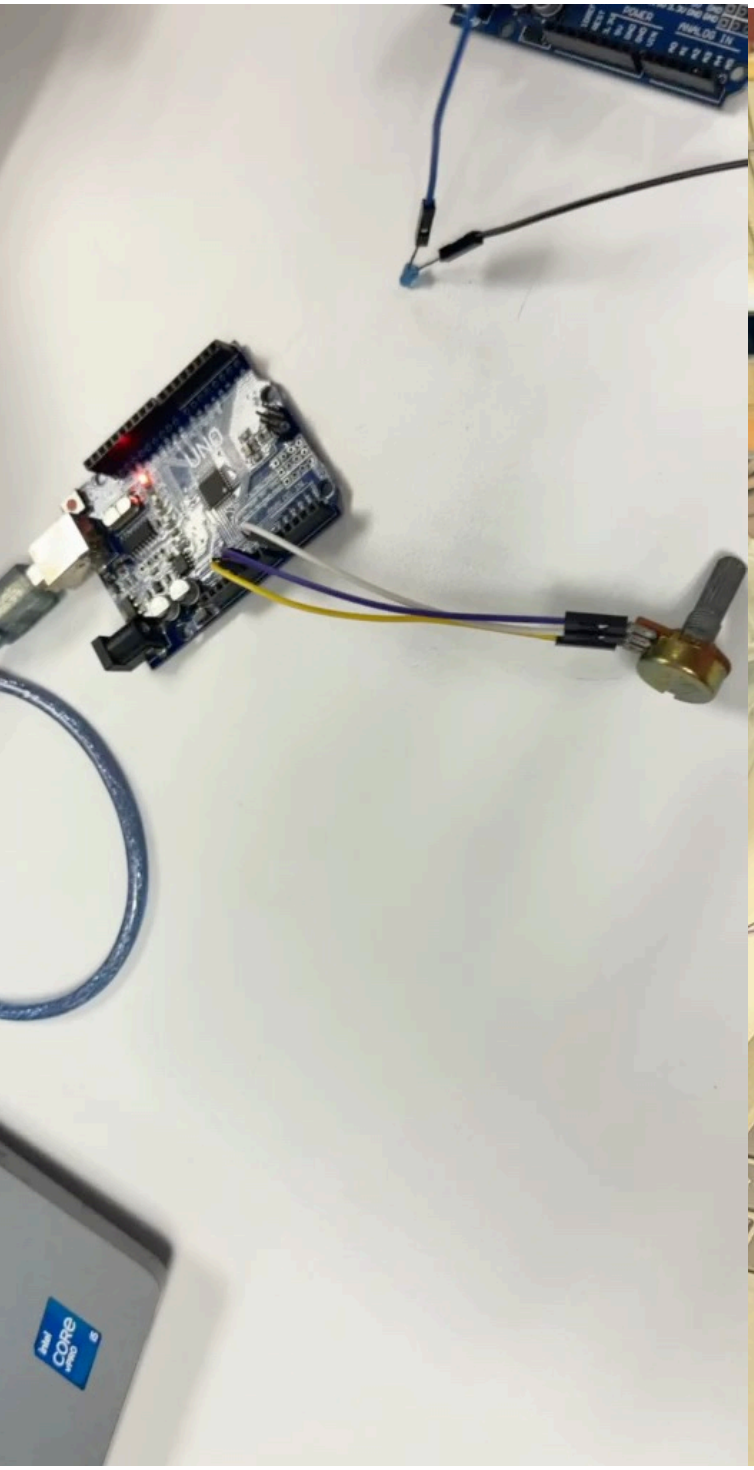
digitalWrite(LED_BUILTIN, LOW); // 4. Apaga el LED
delay(sensorValue); // 5. Se queda apagado el mismo tiempo

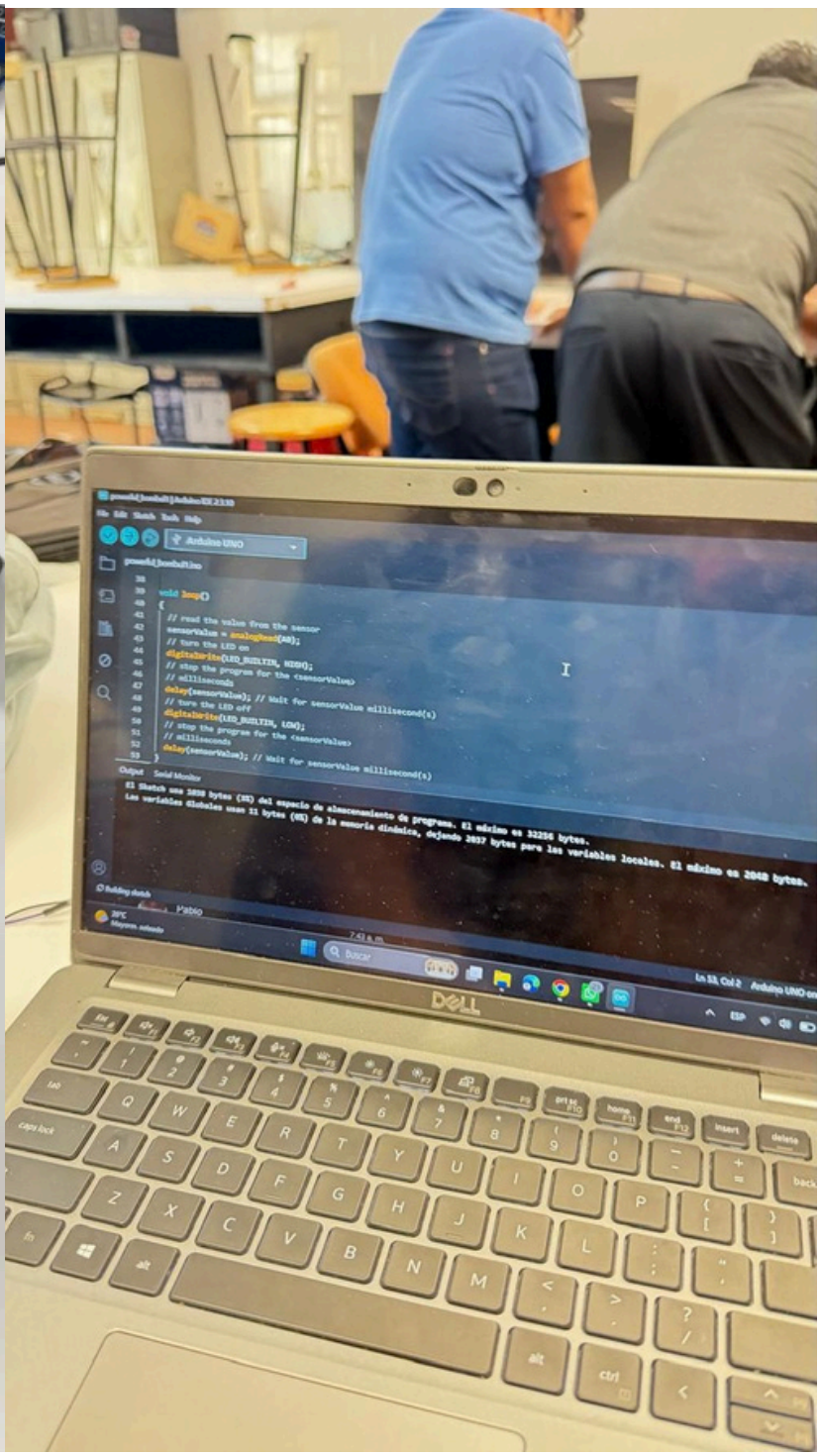
En palabras simples:
Si giras la perilla a la izquierda, `sensorValue` es chiquito (ej. 100), entonces el `delay(100)` es de 0.1 segundos y parpadea rapidísimo.
Si la giras a la derecha, `sensorValue` es grande (ej. 900), entonces `delay(900)` es casi 1 segundo y parpadea lento.

Ya pasaste de una salida digital simple a una *entrada analógica*. Estás controlando el tiempo de parpadeo con hardware.

Para tu reporte pon:

> "Se implementó un control de retardo mediante un potenciómetro conectado a la entrada analógica A0. El valor leído con analogRead() (0-1023) se utiliza como parámetro en la función delay(), permitiendo modificar la frecuencia de parpadeo del LED conectado al pin 13 de forma manual y en tiempo real."





• Botón

Explica

Esta es la práctica 3, la de *Entrada Digital con Botón*. Ya no estás prendiendo nada, ahora estás *LEYENDO* información.

1. El circuito (lo del video)
- *Placa azul Arduino UNO* conectada a tu Huawei.
- *Protoboard blanca:* Ahí está el botoncito negro de 4 patitas.
- 2 cables van del botón al Arduino: uno al *pin 2* y otro a *GND*. El otro cable blanco/café es para darle 5V.

2. El código (tu foto `sketch_sep28a.ino`)

```
const int btnPin = 2; // El botón está en el pin 2

void setup() {
Serial.begin(9600); // Inicia la comunicación con la compu
pinMode(btnPin, INPUT); // Pin 2 será ENTRADA
}

void loop() {
int estado = digitalRead(btnPin); // Lee si es 1 (5V) o 0 (0V)

Serial.print("Estado del boton: ");
Serial.println(estado); // Lo manda a la pantalla

delay(250); // Espera 250ms para no llenar la pantalla de texto
}
```

3. Lo que ves abajo en "Serial Monitor"

```
Estado del boton: 0
Estado del boton: 0
Estado del boton: 1
Estado del boton: 1
```

- *0 = LOW / 0V:* Botón *NO presionado*.
- *1 = HIGH / 5V:* Botón *presionado*.

Cuando presionas el botón en la protoboard, en la pantalla de tu laptop cambia de 0 a 1 instantáneamente. Es un sensor digital, solo tiene dos estados, como un apagador.

Para tu reporte:

> "Se configuró el pin 2 como entrada digital para leer un pulsador. Mediante digitalRead() se obtiene el estado lógico del botón y se envía por comunicación serial a 9600 baudios al Monitor Serie. Se observó el cambio de estado de 0 a 1 al presionar el botón, validando la lectura de entradas digitales."

sketch_sep28a | Arduino IDE 2.3.10

Edit Sketch Tools Help

Arduino UNO

sketch_sep28a.ino

```ino
// Definimos el pin donde está conectado el botón
const int btnPin = 2;

void setup() {
  // Iniciamos la comunicación serie a 9600 baudios
  Serial.begin(9600);

  // Configuramos el pin 2 como ENTRADA
  pinMode(btnPin, INPUT);
}

void loop() {
  // Leemos si entra un 1 (HIGH / 5V) o un 0 (LOW / 0V)
  int estado = digitalRead(btnPin);

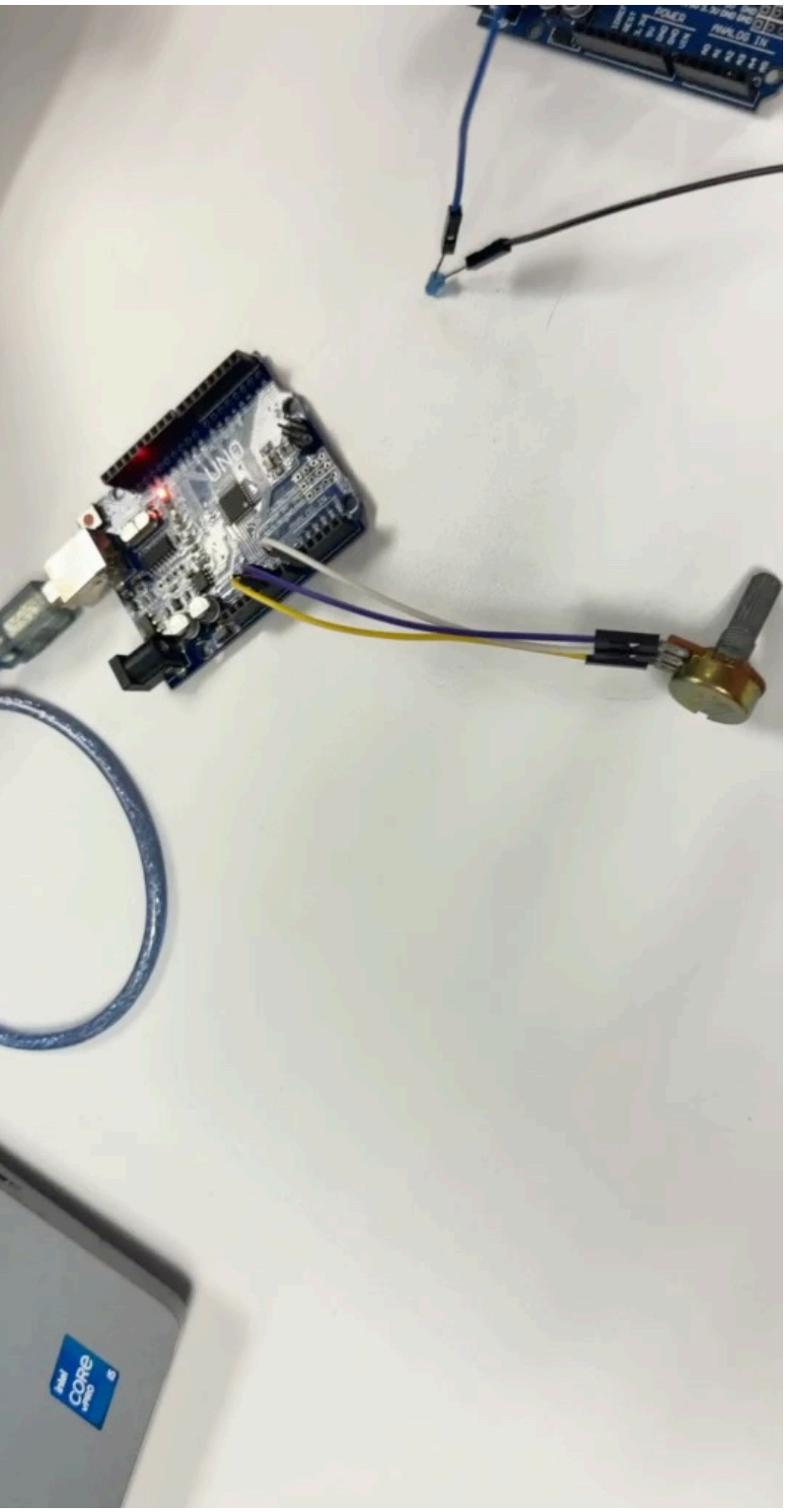
  // Enviamos la lectura al Monitor Serie
  Serial.print("Estado del botón: ");
  Serial.println(estado);

  // Pausa de 250 ms para no saturar la pantalla de texto
  delay(250);
}
```

Output

```
El Sketch usa 2008 bytes (6%) del espacio de almacenamiento de programa. El máximo es 32256 bytes.
Las variables Globales usan 208 bytes (10%) de la memoria dinámica, dejando 1840 bytes para las variables locales. El máximo es 2048 bytes.
```

Ln 18, Col 26 Arduino UNO on COM4 2



- **Serie de lectura digital**

- **Entrada analógica**

Ahora sí, este es el *final de la práctica*, ya juntaste todo. Es la práctica de *Entrada + Salida Digital.*

Lo que hiciste

Ya no solo lees el botón, ahora el botón *controla* el LED.

Tu código `sketch_sep28a.ino` dice:

```
const int btnPin = 2; // Botón en pin 2
const int ledPin = 13; // LED en pin 13 (el integrado del Arduino)

void setup() {
pinMode(btnPin, INPUT); // Pin 2 escucha
pinMode(ledPin, OUTPUT); // Pin 13 manda energía
Serial.begin(9600);
}

void loop() {
int estadoBoton = digitalRead(btnPin); // Lee 0 o 1

if (estadoBoton == HIGH) { // Si está presionado
digitalWrite(ledPin, HIGH); // Prende LED
Serial.println("Encendido");
} else { // Si está suelto
digitalWrite(ledPin, LOW); // Apaga LED
Serial.println("Apagado");
}
}
```

Lo que se ve en tu video

En el *Serial Monitor* abajo sale:

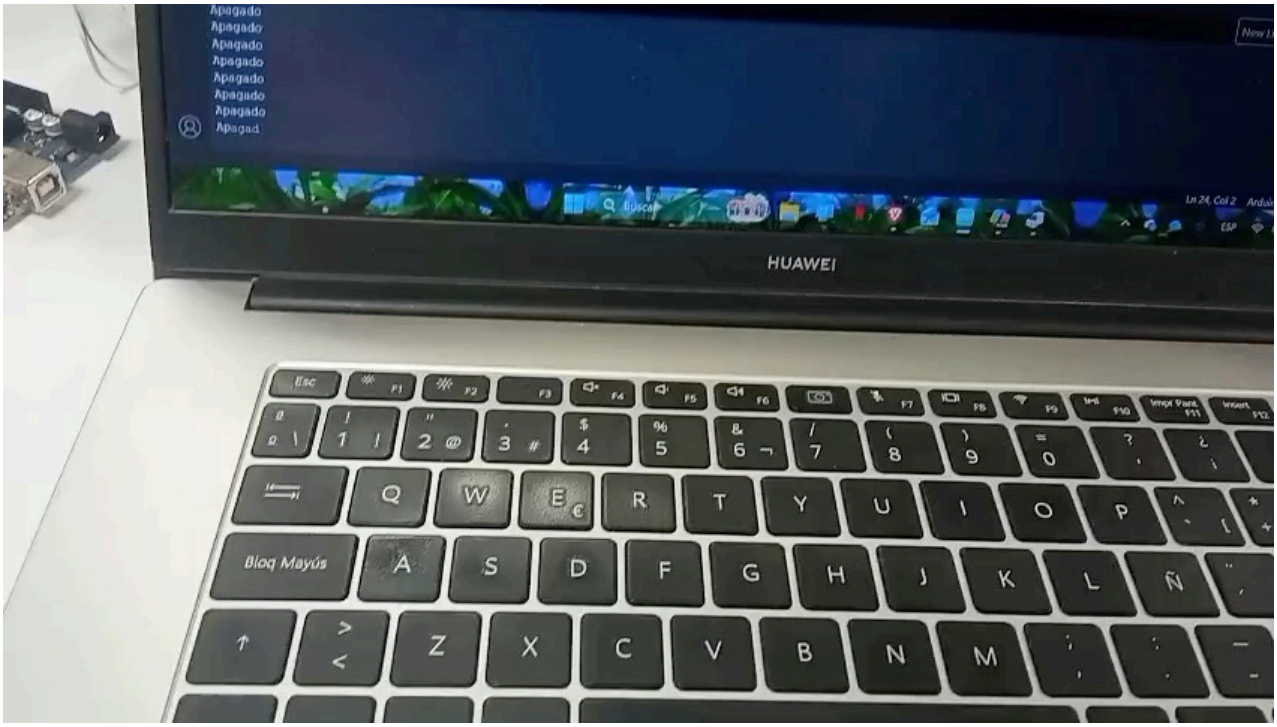
```
Apagado
Apagado
Encendido
Encendido
Apagado
```

Eso pasa porque:
- Cuando *NO presionas* el botón -> `digitalRead` lee `LOW (0)` -> entra al `else` -> apaga el LED del Arduino y escribe "Apagado"
- Cuando *SÍ presionas* el botón de la protoboard -> lee `HIGH (1)` -> entra al `if` -> prende el LED (el foquito naranja que se ve en la placa) y escribe "Encendido"

Es un circuito de control básico. Es la lógica de cualquier interruptor: *SI botón presionado, ENTONCES prende, SI NO, apaga.*

Para tu reporte final pon esto:

> "Se integró una entrada digital (pulsador en pin 2) con una salida digital (LED en pin 13). El estado del botón se evalúa con una estructura condicional if-else. Si el estado es HIGH el LED se enciende, si es LOW se apaga. El estado se visualiza simultáneamente en el Monitor Serie, demostrando el control de actuadores a partir de sensores digitales."



sketch_sep28a | Arduino IDE 2.3.10

File Edit Sketch Tools Help

Arduino UNO

sketch_sep28a.ino

```ino
// Definimos los pines de conexión
const int btnPin = 2;   // Pin donde está conectado el botón
const int ledPin = 13;  // Pin del LED integrado (o LED externo en el pin 13)

void setup() {
  // Configuración de los pines
  pinMode(btnPin, INPUT);   // El pin 2 escuchará si el botón está presionado
  pinMode(ledPin, OUTPUT);  // El pin 13 enviará energía al LED
  Serial.begin(9600);
}

void loop() {
  // Leemos el estado del botón (HIGH si está presionado, LOW si no)
  int estadoBoton = digitalRead(btnPin);

  // Evaluamos la lectura:
  if (estadoBoton == HIGH) {
    digitalWrite(ledPin, HIGH); // Presionado: Enciende el LED
    Serial.println("Encendido");
  } else {
    digitalWrite(ledPin, LOW);  // Suelto: Apaga el LED
    Serial.println("Apagado");
  }
}
```

Output

```
El Sketch usa 1960 bytes (6%) del espacio de almacenamiento de programa. El máximo es 32256 bytes.
Las variables Globales usan 206 bytes (10%) de la memoria dinámica, dejando 1842 bytes para las variables locales. El máximo es 2048 bytes.
```

Ln 24, Col 2 Arduino UNO on COM4 [not connected]

- **Servomotor**

Explica

Este ya es otra cosa, ya pasaste de LED y botones. Este es un *Actuador - Servomotor SG90.*

Es el motorcito azul con la hélice blanca que se ve en tu video.

1. El hardware
- *El motor azul:* Es un servo SG90. No da vueltas completas como un ventilador normal, solo gira de 0° a 180°. Adentro tiene engranes y sabe en qué ángulo está.
- *Los 3 cables:*
- Café/Negro -> GND
- Rojo -> 5V
- Naranja -> Señal (Pin 9)
- Está conectado directo a tu Arduino UNO genérico (el azul con letras UNO). El USB le da suficiente energía para mover esa hélice.

2. El código que se ve en tu Dell

Este es el ejemplo `Sweep`, el que viene en Arduino:

```
#include <Servo.h> // 1. Incluye la librería para controlar servos

int pos = 0;
Servo servo_9; // 2. Crea un objeto servo

void setup() {
servo_9.attach(9, 500, 2500); // 3. Le dices que el servo está en el pin 9
}

void loop() {
for (pos = 0; pos <= 180; pos += 1) { // 4. Va de 0 a 180 grados
servo_9.write(pos); // Manda al servo a ese ángulo
delay(15); // Espera 15ms para que llegue
}
for (pos = 180; pos >= 0; pos -= 1) { // 5. Regresa de 180 a 0
servo_9.write(pos);
delay(15);
}
}
```

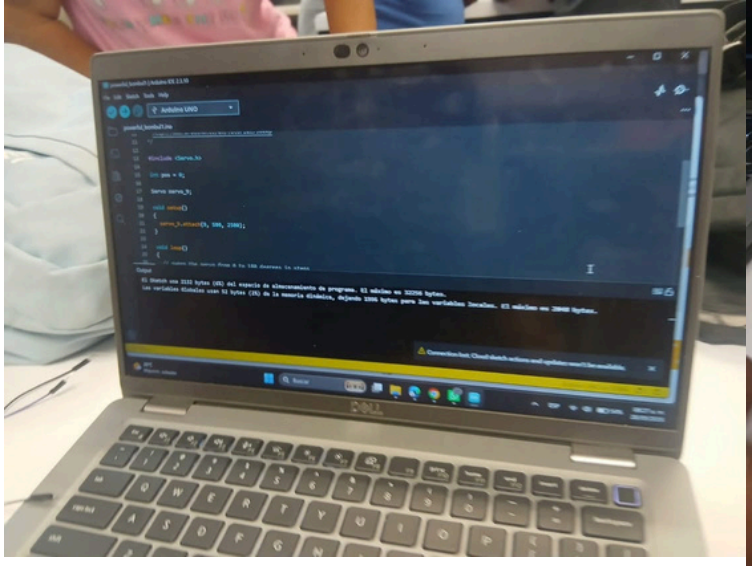
3. Lo que pasa en el video

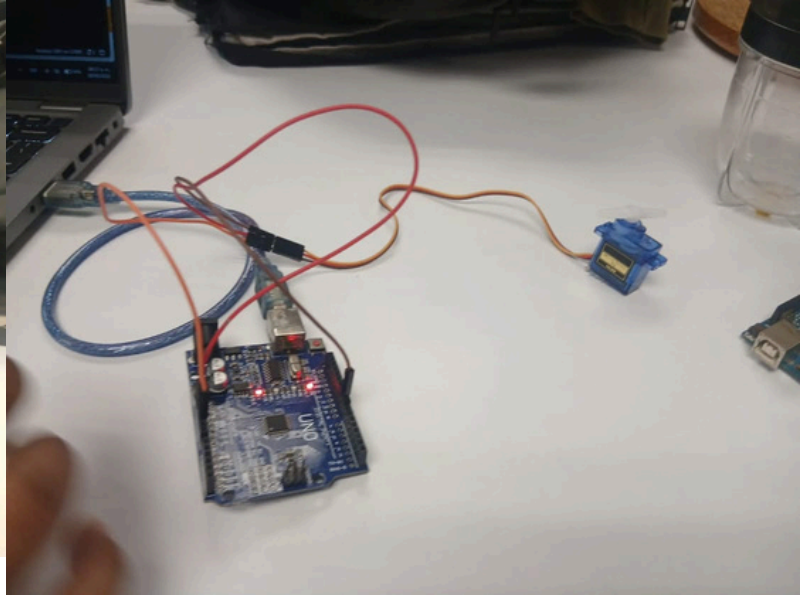
El `for` hace que el Arduino le mande una señal PWM al cable naranja. Esa señal le dice al servo "vete a 10°, ahora a 11°, ahora a 12°...".

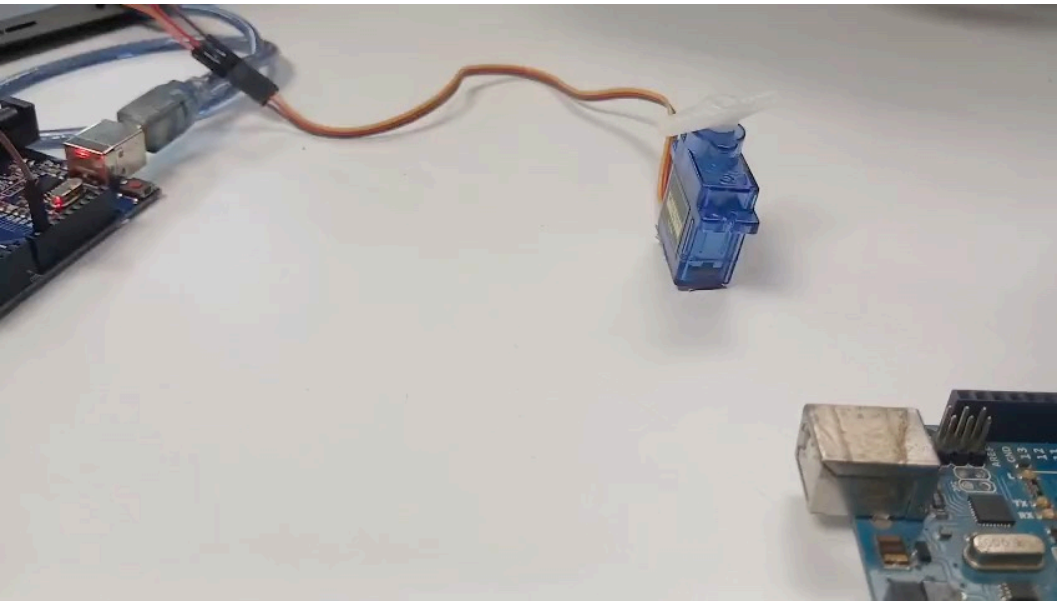
Por eso ves que la hélice va lenta de un lado a otro, de 0 a 180 y de regreso. No es un giro libre, es un barrido controlado.

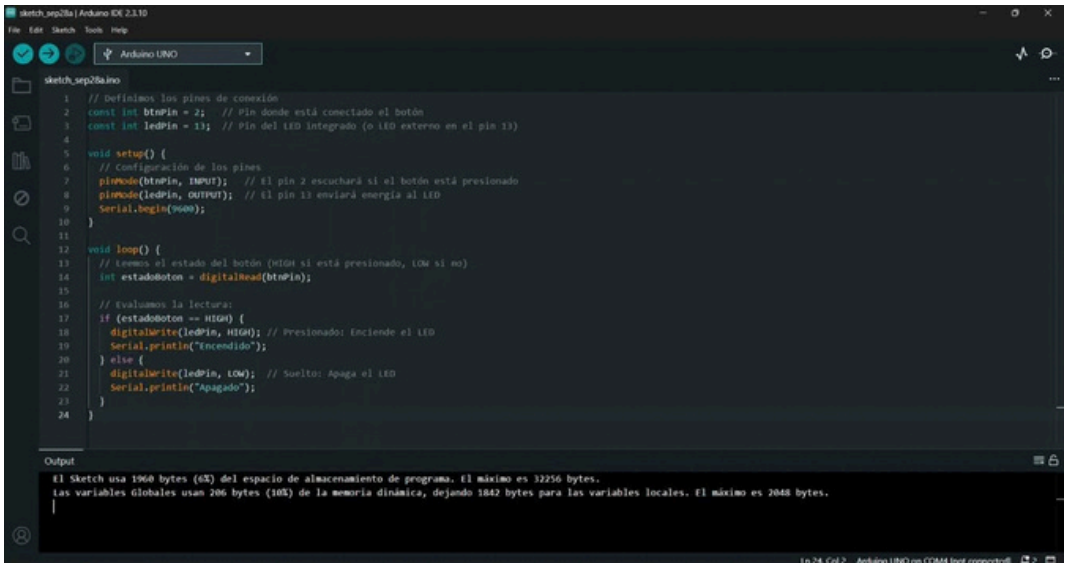
Para tu reporte final pon:

> "Se controló un servomotor SG90 mediante la librería Servo.h conectado al pin 9. La función attach() configura el pin con pulsos de 500 a 2500 microsegundos y write() posiciona el eje entre 0 y 180 grados. El programa realiza un barrido continuo en ambos sentidos con retardo de 15 ms, demostrando el control de posición angular por modulación de ancho de pulso (PWM)."








sketch_sep28a | Arduino IDE 2.3.10
File Edit Sketch Tools Help
Arduino UNO
sketch_sep28a.ino
// Definimos los pines de conexión
const int btnPin = 2; // Pin donde está conectado el botón
const int ledPin = 13; // Pin del LED integrado (o LED externo en el pin 13)
void setup() {
// Configuración de los pines
pinMode(btnPin, INPUT); // El pin 2 escuchará si el botón está presionado
pinMode(ledPin, OUTPUT); // El pin 13 enviará energía al LED
Serial.begin(9600);
}
void loop() {
// Leemos el estado del botón (HIGH si está presionado, LOW si no)
int estadoBoton = digitalRead(btnPin);
// Evaluamos la lectura:
if (estadoBoton == HIGH) {
digitalWrite(ledPin, HIGH); // Presionado: Enciende el LED
Serial.println("Encendido");
} else {
digitalWrite(ledPin, LOW); // Suelto: Apaga el LED
Serial.println("Apagado");
}
}
Output
El Sketch usa 1960 bytes (6%) del espacio de almacenamiento de programa. El máximo es 32256 bytes.
Las variables Globales usan 206 bytes (10%) de la memoria dinámica, dejando 1842 bytes para las variables locales. El máximo es 2048 bytes.